Understanding `this` Keyword in Java

Introduction

The `this` keyword in Java is a reference variable that refers to the current object. It is mainly used to eliminate ambiguity between instance variables and parameters, invoke current class methods or constructors, and return the current class instance.

Uses of `this` Keyword

1. Referring to Instance Variables

When instance variables have the same name as method parameters, `this` can be used to differentiate them.

```
class Student {
    String name;

    Student(String name) {
        this.name = name; // Using `this` to differentiate instance
variable and parameter
    }

    void display() {
        System.out.println("Student Name: " + this.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul");
        s1.display(); // Output: Student Name: Rahul
    }
}
```

2. Invoking Current Class Method

The `this` keyword can be used to explicitly call the current class method.

```
class Example {
    void show() {
        System.out.println("Show method is called");
    }

    void display() {
        this.show(); // Calling show() using `this`
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj = new Example();
        obj.display(); // Output: Show method is called
    }
```

```
}
```

3. Invoking Current Class Constructor

We can use `this()` to call another constructor within the same class.

```
class Person {
    String name;
    int age;

    Person() {
        this("Unknown", 0); // Calling another constructor
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Person p1 = new Person();
        p1.display(); // Output: Name: Unknown, Age: 0
    }
}
```

4. Returning Current Class Instance

The `this` keyword can be used to return the current instance of the class.

```
class Example {
    Example getInstance() {
        return this;
    }

    void show() {
        System.out.println("Returning current instance");
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj = new Example().getInstance();
        obj.show(); // Output: Returning current instance
    }
}
```

5. Passing `this` as Argument

The `this` keyword can be passed as an argument in a method call.

```
class A {
```

```
    void method1(A obj) {
        System.out.println("Method is called using this keyword");
    }

    void method2() {
        method1(this); // Passing current instance
    }
}

public class Main {
    public static void main(String[] args) {
        A obj = new A();
        obj.method2(); // Output: Method is called using this keyword
    }
}
```

6. Passing `this` as Argument in Constructor

`this` can also be used to pass the current instance to another constructor.

```
class B {
    B(A obj) {
        System.out.println("Constructor is called with this keyword");
    }
}

class A {
    void show() {
        B obj = new B(this);
    }
}

public class Main {
    public static void main(String[] args) {
        A obj = new A();
        obj.show(); // Output: Constructor is called with this keyword
    }
}
```

Common Interview Questions on `this` Keyword

1. What is the purpose of the `this` keyword in Java?

Answer: The `this` keyword refers to the current instance of the class. It is used to eliminate ambiguity between instance variables and parameters, call the current class's constructor, return the current instance, and pass it as an argument.

2. Can `this` be used inside a static method?

Answer: No, `this` cannot be used inside a static method because `this` refers to an instance, and static methods do not belong to any particular instance.

3. How can you call a constructor within another constructor?

Answer: By using `this()`, we can call a constructor within another constructor of the same class.

4. What happens if we return `this` from a method?

Answer: Returning `this` from a method allows method chaining and provides the current instance.

Example:

```
class Test {
    Test get() {
        return this;
    }
}
```

5. Can `this` be assigned a new value?

Answer: No, `this` is a final reference and cannot be assigned a new value.

```
class Test {
    void change() {
        // this = new Test(); // Compilation error
    }
}
```

Conclusion

The `this` keyword is a powerful concept in Java that helps manage instance variables, invoke constructors and methods, and return the current instance. Understanding its usage is crucial for Java developers, especially in object-oriented programming and interview scenarios.